

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো

(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ

(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাস

(Indian Institute of Technology, Madras)

সপ্তাহ – 05

বক্তৃতা – 22

সম্পূর্ণভাবে পক্ষপাতশূন্য শিডিউলিং

(Completely Fair Scheduling)

এই ভিডিও বক্তৃতাতে, আমরা সম্পূর্ণভাবে পক্ষপাতশূন্য শিডিউলারটি (completely fair scheduler) দেখব। লিনাক্স কার্নেলের সাম্প্রতিক সংস্করণের (latest version) default শিডিউলার হল সম্পূর্ণভাবে পক্ষপাতশূন্য শিডিউলার (completely fair scheduler) বা CFS শিডিউলার।

(স্লাইড সময় : 00:29 দেখুন)

CFS শিডিউলার থেকে লিনাক্স কার্নেলে অন্তর্ভুক্ত হয়েছে 2.6.23 সংস্করণ থেকে এবং 2007 সাল থেকে শিডিউলিং অ্যালগরিদম হিসাবে ব্যবহার করা হয়েছে। এটির ভিত্তি কন কলিভাসের (Con Kolivas) রোটেটিং স্টেয়ারকেস ডেডলাইন সিডিউলার (Rotating Staircase Deadline Scheduler) এর উপর। $O(1)$ শিডিউলারের থেকে CFS শিডিউলারের সুবিধা এই যে এখানে কোন হিউরিস্টিক্স ব্যবহার করা হয় না এবং I/O bound ও CPU bound প্রসেসগুলিকে খুব সুন্দর ভাবে পরিচালনা করা হয়। বস্তুত interactive এবং non-interactive বা batch প্রসেসগুলি এই শিডিউলারে খুব সহজেই খাপ খেয়ে যায়। আমরা CFS শিডিউলারের খুব সংক্ষিপ্ত বিবরণ দেখব।

নাম থেকেই বোঝা যাচ্ছে যে CFS শিডিউলার বা completely fair scheduler এর লক্ষ্য হল প্রসেসগুলির মধ্যে CPU এর সময় নিরপেক্ষ ভাবে বা সমভাবে বন্টন করা।

(স্লাইড সময় : 01:34 দেখুন)

অন্য ভাবে বলা যায় যে, যদি সিস্টেমের মধ্যে বা রেডি কিউতে N টি প্রসেস উপস্থিত থাকে যারা শিডিউলিংয়ের জন্য অপেক্ষা করছে তাহলে প্রতিটি প্রসেস CPU সময়ের 100 ভাগ N শতাংশ পাবে। এটাই আদর্শ নিরপেক্ষতা। এই আদর্শ নিরপেক্ষ শিডিউলিংয়ের একটি ছোট থিয়োরিটিকাল উদাহরণ দেখা যাক। মনে করা যাক A, B, C ও D চারটি প্রসেস আছে যাদের burst time যথাক্রমে 8 মিলিসেকেন্ড, 4 মিলিসেকেন্ড, 16 মিলিসেকেন্ড ও 4 মিলিসেকেন্ড। আমরা এবার সময়কে 4 মিলিসেকেন্ডের টুকরো (time quanta) তে ভেঙ্গে নিয়ে দেখব যে কি ভাবে আদর্শ নিরপেক্ষ শিডিউলিং (ideal fair scheduling) কাজ করে। আদর্শ নিরপেক্ষ শিডিউলিং 4 মিলিসেকেন্ড সময় শেষে এমন হতে হবে যে রেডি কিউতে যে প্রসেসগুলি আছে তারা একই পরিমাণ clock cycle সময় এক্সিকিউট করেছে। আমরা যদি এই প্রথম 4 মিলিসেকেন্ডের সময়কালটি দেখি, এখানে 4টি প্রসেস আছে। সুতরাং প্রতিটি প্রসেস CPU সময়ের 4 ভাগ 4 অর্থাৎ 1 মিলিসেকেন্ড পাবে। সুতরাং A, B, C ও D প্রত্যেকে 1 মিলিসেকেন্ড করে এক্সিকিউট করবে। একই ভাবে দ্বিতীয় চক্রেও চারটি প্রসেস আছে, সুতরাং প্রতিটি প্রসেস সময় অংশের (time slice) সমান ভাগ পাবে। অর্থাৎ প্রতিটি প্রসেস আবার 1 মিলিসেকেন্ড করে এক্সিকিউট করবে। সুতরাং সব মিলিয়ে A এক্সিকিউট করেছে 2 মিলিসেকেন্ড, B করেছে 2 মিলিসেকেন্ড, C করেছে 2 মিলিসেকেন্ড এবং D এক্সিকিউট করেছে 2 মিলিসেকেন্ড। একই ভাবে তৃতীয় ও চতুর্থ চক্র দুইটি হবে। অতএব চতুর্থ চক্রের শেষে B এবং D প্রসেস দুইটি সম্পন্ন হয়ে যাবে। এরপরে কি হবে?

(স্লাইড সময় : 03:32 দেখুন)

B এবং D প্রসেস দুইটি সম্পন্ন হওয়ার পরে রেডি কিউতে A এবং C দুইটি প্রসেস পড়ে থাকে, কিন্তু time quantum এখনও 4 মিলিসেকেন্ড। সুতরাং এবার প্রতিটি প্রসেস 4 ভাগ 2 অর্থাৎ 2 মিলিসেকেন্ড করে CPU সময় পাবে। কাজেই প্রসেস A এবং C উভয়েই 2 মিলিসেকেন্ড করে এক্সিকিউট করবে। অর্থাৎ দুইটি প্রসেসই মোট 8 মিলিসেকেন্ড এক্সিকিউট করেছে। সুতরাং A এর এক্সিকিউশন সম্পূর্ণ হয়েছে।

(স্লাইড সময় : 04:16 দেখুন)

এখন, শেষ অংশে রেডি কিউতে শুধুমাত্র C প্রসেসটি রয়েছে। অতএব C প্রসেসটিকে সম্পূর্ণ 4 মিলিসেকেন্ডের time quantum দেওয়া হবে। সুতরাং C একটি চক্রে 4 মিলিসেকেন্ড ও পরবর্তী চক্রে 4 মিলিসেকেন্ড এক্সিকিউট করে তার burst time সম্পূর্ণ করবে। অতএব যেটা দেখা গেল এই আদর্শ শিডিউলিং প্রক্রিয়াতে শিডিউলার প্রতিটি প্রসেসের মধ্যে সমভাবে সময় বন্টন করার প্রচেষ্টা করে যার ফলে প্রতিটি প্রসেস asymptotic ভাবে একই

সময় CPU তে এক্সিকিউট করে। অতএব দেখা যাচ্ছে যে সবকটি প্রসেস প্রথমে 4 মিলিসেকেন্ড, তারপরে 6 মিলিসেকেন্ড, তারপরে 8 মিলিসেকেন্ড, এই ভাবে CPU তে এক্সিকিউট করে। কিভাবে তাহলে CFS শিডিউলারে এই আদর্শ নিরপেক্ষ শিডিউলিং (ideal fair scheduling) অন্তর্ভুক্ত করা হয়?

(স্লাইড সময় : 05:21 দেখুন)

এটি করা হয় যাকে বলে আপাত চল সময় (virtual run time) এর সাহায্যে। প্রতিটি প্রসেসের প্রসেস কন্ট্রোল ব্লক (process control block) (PCB) তে vruntime বা virtual run time এর অন্তর্ভুক্তি (entry) থাকে। প্রতিটি শিডিউলিংয়ের সময় যদি দেখা যায় একটি প্রসেস t মিলিসেকেন্ড এক্সিকিউট করেছে তাহলে তার vruntime t মিলিসেকেন্ড বাড়িয়ে দেওয়া হয়। সুতরাং একটি প্রসেসের vruntime একই ভাবে বাড়তে থাকে।

(স্লাইড সময় : 05:51 দেখুন)

এখন CFS এর মূল উদ্দেশ্য হল যখনই context switch হবে তখনই যে কার্যটির সবচেয়ে কম vruntime আছে সেটিকে বেছে নেওয়া হবে। এটি min underscore vruntime নামক একটি চলরাশি (variable) দ্বারা পরিচালনা করা হয়, যেটি ক্ষুদ্রতম virtual run time সম্পন্ন প্রসেসের নির্দেশক (pointer)। এই প্রসেসটির প্রয়োজনীয় dynamic time slice নির্ণয় করা হয় এবং high resolution timer টি এই time slice দ্বারা প্রোগ্রাম করা হয়। প্রসেসটি এরপর CPU তে এক্সিকিউট করা শুরু করে। যখন একটি interrupt ঘটে তখন যদি কোন আরও ছোট run time সম্পন্ন কোন প্রসেস থাকে তাহলে আবার context switch হয়। অতএব দেখা যাচ্ছে যে যতক্ষণ না ক্ষুদ্রতর run time এর প্রসেস পাওয়া যাচ্ছে ততক্ষণ আগের নির্বাচিত প্রসেসটি এক্সিকিউট করতে থাকে।

(স্লাইড সময় : 06:51 দেখুন)

বিভিন্ন run time বিশিষ্ট এতগুলি প্রসেস বা টাস্কে নিয়ন্ত্রণ করার জন্য CFS শিডিউলার আমাদের এতক্ষণ দেখা শিডিউলারগুলির মতো রেডি কিউ ব্যবহার করে না, বরং red black tree নামক একটি data structure ব্যবহার করে। Red black tree বা rb tree এর প্রতিটি node একটি এক্সিকিউট করতে পারে এমন একটি টাস্কে নির্দেশ করে। Node গুলিকে তাদের virtual run time অনুসারে সাজান হয়। বাঁদিকের node গুলির virtual run time বা vruntime ডানদিকেরগুলির অপেক্ষা কম। এখানে দেখা যাচ্ছে যে প্রতিটি node এক একটি টাস্ক এবং node এর উপর লিখিত সংখ্যাগুলি সেই নির্দিষ্ট টাস্কের vruntime। এটা দেখা যাচ্ছে যে বাম দিকের প্রতিটা টাস্কের virtual run time ডান দিকেরগুলির তুলনায় কম। সুতরাং এই rb tree এর সর্বাপেক্ষা বাঁদিকের node টির virtual run time সবচেয়ে কম। অতএব শিডিউলার পরবর্তী এক্সিকিউশনের জন্য এই সবচাইতে কমের virtual run time এর টাস্কটিকে বেছে নেবে। এই টাস্কটিকে নির্দিষ্ট করার জন্য দুইটি সম্ভাব্য উপায় আছে। একটি হল যতদূর সম্ভব বাঁদিকে গিয়ে যতক্ষণ না একটি leaf পাওয়া যাচ্ছে tree টি traverse করা বা অন্যথায় min

underscore vruntime মত একটি pointer রাখা যা সর্বদা সব থেকে বাঁদিকের node টি নির্দেশ করবে। সুতরাং যখনই শিডিউলারের context switch করার প্রয়োজন হবে সেটি যে node কে min vruntime নির্দেশ করছে সেই প্রসেসটি বেছে নেবে। স্বভাবতই এটি হবে ন্যূনতম virtual run time সম্পন্ন টাস্ক।

(স্লাইড সময় : 08:48 দেখুন)

এই ন্যূনতম virtual run time বাছাইয়ের কাজটি $O(1)$ এ হতে পারে এবং সেই কারণে rb tree তে কয়টি প্রসেস আছে তার উপর নির্ভর করে না। Time slice শেষ হলে যদি দেখা যায় যে বর্তমানে যে প্রসেসটি এক্সিকিউট করছে সেটি কোন I/O এর জন্য অপেক্ষা করছে না বা তার কার্য শেষ হয়নি তাহলে যতক্ষণ সেটি CPU তে এক্সিকিউট করেছে তার উপর ভিত্তি করে সেটির নতুন virtual run time নির্ণয় করা হয়। এই নতুন virtual run time ধরে সেটিকে আবার tree তে প্রবেশ করানো হয়। অন্যভাবে বলা যায় যে rb tree এর সর্বাপেক্ষা বাঁদিকের node থেকে একটি প্রসেসকে বেছে নাওয়া হয়, সেটিকে CPU তে এক্সিকিউট করা হয়, ধরা যাক t মিলিসেকেন্ডের জন্য এবং তারপরে সেটির virtual run time t মিলিসেকেন্ড বাড়িয়ে দেওয়া হয়। এবার এটি rb tree এর বাঁদিকের বদলে ডানদিকের কোন মাঝামাঝি স্থানে প্রবেশ করবে। অতএব যত virtual run time বাড়তে থাকে একটি প্রসেস তত বাম থেকে ডানদিকে সরতে থাকে। এর থেকে এটি নিশ্চিত করা যায় যে সবকটি প্রসেসই কখনো না কখনো এই tree তে সর্বাপেক্ষা কম virtual run time পাবে এবং এক্সিকিউট করার সুযোগ পাবে। এরফলে starvation এড়ানো যাবে।

(স্লাইড সময় : 10:24 দেখুন)

এখন প্রশ্ন হল লিনাক্স কার্নেল কেন CFS শিডিউলারের জন্য red black tree ই ব্যবহার করে? একটি স্পষ্ট কারণ হল rb tree স্বপ্রতীয়মান বা self balancing। Self balancing এর ফলে কোন path এর দৈর্ঘ্যই অন্য একটি path এর দৈর্ঘ্যের দ্বিগুণ হয় না। এই কারণে operation গুলি $O(\log n)$ হবে, ফলে tree টিতে insert বা delete operation দ্রুত এবং কার্যকরী হয়।

(স্লাইড সময় : 10:55 দেখুন)

এখন দেখা যাক CFS শিডিউলারে অগ্রাধিকার (priority) কি ভাবে সম্পাদন করা যায়? CFS শিডিউলার $O(1)$ শিডিউলারের মতো কোন স্বতন্ত্র অগ্রাধিকার (exclusive priority) বিশিষ্ট কিউ ব্যবহার করে না, বরং এটি শুধুমাত্র virtual run time নির্ণয় করার জন্য অগ্রাধিকার ব্যবহার করে। যেমন, যদি একটি প্রসেস t মিলিসেকেন্ড run করে থাকে তাহলে nice এর মানের weight এ সেই প্রসেসটির virtual run time t মিলিসেকেন্ড বাড়িয়ে দেওয়া হয়, এবং এটি প্রসেসটির স্থির অগ্রাধিকার (static priority) ভিত্তিতে হয়। সুতরাং নিম্নতর অগ্রাধিকারের মান উচ্চতর অগ্রাধিকারের তুলনায় দ্রুততর বৃদ্ধি পায়। অতএব যেটা করা হচ্ছে সেটি

হল একটি প্রসেস কতক্ষণ এক্সিকিউট করবে তার উপর একটি weight দিয়ে প্রসেসটি কতক্ষণ এক্সিকিউট করবে সেটি স্বরাশ্রিত (accelerate) বা মন্দীভূত করা হচ্ছে। সুতরাং CFS scheduling algorithm এ এই weight ব্যবহার করেই CFS scheduling algorithm এ অগ্রাধিকার নির্ণয় করা হয়।

(স্লাইড সময় : 12:01 দেখুন)

এবার আমরা দেখব যে CFS শিডিউলার কি ভাবে I/O bound এবং CPU bound প্রসেসগুলির মধ্যে প্রভেদ করতে পারে। বস্তুত এই প্রভেদ করার কাজটি খুবই কার্যকারী ভাবে করা হয়। এটি এই ঘটনার উপর ভিত্তি করে করা হয় যে I/O bound প্রসেসগুলির CPU burst এর সময় খুব কম হয়, ফলে এদের vruntime এর মান উল্লেখযোগ্য পরিমাণে বৃদ্ধি পায় না। এই কারণে এরা প্রায়ই rb tree এর বাঁদিকের অংশে চলে আসে এবং অন্য প্রসেসের তুলনায় বেশি ঘন ঘন এক্সিকিউট করার সুযোগ পায়। এর কারণ হল যেমন আমরা দেখেছি যে সময় যাওয়ার সাথে সাথে CFS শিডিউলারের প্রসেসগুলিকে rb tree এর বাঁদিকের node থেকে তুলে, এক্সিকিউট করে, ডানদিকে নিয়ে আসা হয়, অর্থাৎ শিডিউলারের প্রতিটি প্রসেস ডানদিকে সরতে থাকে। যেহেতু I/O bound প্রসেসগুলির vruntime অতি অল্প পরিমাণে বৃদ্ধি পায়, এগুলি tree এর ডানদিকের একদম শেষ প্রান্তে না গিয়ে tree এর বাঁদিকের অংশেই থাকে। এর ফলে শীঘ্রই এটির vruntime সর্বাপেক্ষা কম হয় এবং এটি পুনরায় CPU তে এক্সিকিউট করার সুযোগ পায়। এখান I/O bound প্রসেসের ছোট vruntime এর দ্বিতীয় প্রভাব হিসাবে, এটি CPU- এ চালানোর জন্য একটি বড় time slice দেওয়া হয়। সুতরাং আমরা দেখলাম যে I / O bound এবং CPU bound প্রসেসগুলো CFS অ্যালগরিদম দ্বারা বেশ স্বতন্ত্রভাবে পৃথক করা হয়।

(স্লাইডটাইপ পড়ুন: 13:50)

যখন একটি নতুন প্রক্রিয়া তৈরি হয়, এটি লাল কালো রঙের tree তে যোগ হয়। এখন এটির প্রাথমিক মান min_vruntime এর সাথে শুরু হয়, tree টির সবচাইতে বামদিকের নোড থাকে এবং এটি নিশ্চিত করে যে এটি খুব দ্রুত execute হবে। সুতরাং, এটি একটি ইন্টারেক্টিভ বা একটি CPU bound প্রসেসের উপর নির্ভর করে execution এর সময় নির্বাচিত হয়, RB tree এর মধ্যে তার অবস্থান পরিবর্তিত হতে পারে। এটা CFS শিডিউলারের একটি সংক্ষিপ্ত ভূমিকা ছিল, যা লিনাক্স কার্নেলের বর্তমান সংস্করণে ডিফল্ট নির্ধারক হিসাবে বিবেচিত হয়। ধন্যবাদ।